

CS2001: Data Structures (Spring 2025)

Semester Project

(Deadline: 05th May 2025, 11:59 PM)

Project groups: This project can be done within a group of two (2) students. There is no restriction on the selection of group members. Students are allowed to make groups according to their preferences. However, the group members must belong to the same **data structure section**.

Submission: All submissions **MUST** be uploaded on Google Classroom. Solutions sent to the emails will not be graded. To avoid last minute problems (unavailability of Google Classroom, load shedding, Internet down etc.), you are strongly advised to start working on the project from day one.

Combine all your work (i.e. Report& .cpp solution folder) in one .zip file. Submit zip file on Google Classroom within given deadline. If only .cpp file is submitted, it will not be considered for evaluation.

Deadline: Deadline to submit project is **05th May 2025, 11:59 PM**. No submission will be considered for grading outside Google Classroom or after **05th May 2025, 11:59 PM**. Correct and timely submission of project is the responsibility of every group; hence no relaxation will be given to anyone.

Plagiarism: You are encouraged to consult online resources for learning and guidance, but directly copying code or written content from the internet, peers, or any other sources is strictly prohibited. Any form of plagiarism, whether in the code or the report, will result in direct zero for the entire project.

Semester Project: Zombie Apocalypse Survival Game

Project Description

In this semester project, you will apply your knowledge of data structures in an engaging and challenging scenario. You will design and implement a **console-based survival game** in C++, where a player must escape a zombie-infested city. The city layout, zombie waves, item collection, and escape pathfinding must be managed using **graphs, queues, and linked lists**, respectively.

Requirements:

Develop a 2D Console-Based Survival Game

Using C++ without any external graphics library. Create a Text-Based User Interface (TUI) using ASCII art.

- The city map is shown using characters (e.g., **#** = wall, **P** = player, **Z** = zombie, **S** = safe zone, ***** = item).
- The player must navigate to a safe zone while avoiding zombies and collecting survival items.

Data Structures Integration:

- **Graph for City Map and Navigation:**
 - Each cell in the map is a **node** in the graph.
 - Edges represent possible paths (up, down, left, right).
 - Implement pathfinding using **Dijkstra's algorithm** or **BFS** for auto-mode or zombie path planning.
- **Queue for Zombie Waves:**
 - Use a **queue** to manage zombie spawning and movement.
 - Zombies are added to the queue and processed in each game turn.
- **Linked List for Collected Items:**
 - Maintain a **singly linked list** to track collected items like medkits, food, and weapons.
 - Each node contains item name, value, and effect on score/health.

Game Logic:

- Two Game Modes:
 - **Manual Mode:** Player controls character using arrow keys.
 - **Automated Mode:** User selects start and end points; game auto-navigates player using shortest path.
- Implement game rules:
 - If a zombie reaches the player, game over.
 - If player reaches the safe zone, player wins.
 - Player collects items along the way for bonus score or health.
 - Zombies move toward the player (if logic permits) or randomly using the graph edges.

Scoring System:

- Formula example:
$$\text{Score} = (\text{Items Collected Value} * 10) - (\text{Zombies Encountered} * 5) + (\text{Distance Traveled} * 2)$$
- You can design and explain your own formula.
- Save high scores in a file with player name.

User Interface (TUI):

- Show a menu: Start Manual / Start Auto / Exit
- Map is redrawn on every move (illusion of animation).
- Display health, score, collected items, and game status (win/loss).

Testing and Optimization:

- Ensure code compiles and runs on standard C++ compilers.
- Test for:
 - Boundary conditions (invalid moves, walls, zombie collision).
 - Smooth zombie movements and spawn timing.
 - Correct functioning of data structures.

Documentation:

- Add comments to all functions.
- Describe how each data structure is used.
- Create a Report (**PDF**) summarizing:
 - Code design
 - Game rules
 - Data structures usage
 - Screenshot of game in play
 - List of known bugs (if any)

Sample Map Layout (TUI Example)

```
=====
|                                     |
|               ZOMBIE APOCALYPSE SURVIVAL               |
|=====|=====|=====|
| [1] Manual Mode | [2] Automated Mode | [3] Exit        |
|=====|=====|=====|
| Health: 100    | Score: 0           | Inventory: [Medkit, Food] |
|=====|=====|=====|
| Map:                                                  |
|#####|
| #P...*...#...Z...#...*...#|
| #.####.####.Z####.####|
| #.#...#...#...#...#|
| #.#.####.#.Z##.#...#*...#|
| #.#...#...#...#...#|
|#####|
|=====|
| Legend:|
| P = Player   Z = Zombie   S = Safe Zone   * = Item   # = Wall/Obstacle|
|=====|
```

Top Banner shows the game title.

Menu Line lets the player choose Manual, Automated, or Exit.

Status Line displays current Health, Score, and collected Inventory.

Map Area is a 2D ASCII grid:

- Walls (#) form the city blocks.
- P marks the player's current position.
- Z are zombies queued to move.
- * are supplies to collect.
- S (in Automated Mode) is the safe-zone goal.

The map is **redrawn each turn** to animate movement.

*** Best of Luck ***